

MÓDULO 8: SEGURIDAD

SECURITY MISCONFIGURATION

OWASP TOP 10

BIG school

Security Misconfiguration

Índice

- ¿Qué vamos a ver?
- ¿Qué es Security Misconfiguration?
- ¿En qué consiste conceptualmente?
- Ejemplos típicos
- ¿Qué puede hacer alguien que explote una misconfiguration?
- ¿Cómo detectarlo?
- ¿Cómo prevenirlo?
- ¿Cómo mitigarlo cuando ya está en producción?
- Checklist operativo rápido
- Conclusiones



Security Misconfiguration

BIG school

¿Qué vamos a ver?

- Errores en instalación, configuración o mantenimiento que dejan sistemas vulnerables por defecto.
- Afecta servidores, bases de datos, contenedores y cloud.
- Veremos definición, causas, ejemplos, impacto, detección, prevención y mitigación práctica.

¿Qué es Security Misconfiguration?

Un sistema se despliega con configuraciones inseguras o por defecto.

- No es un bug de código: el entorno o sus settings facilitan el abuso.
- La configuración forma parte del producto: si es insegura, el producto también lo es.

¿En qué consiste conceptualmente?

Un parte incluye endpoints, pannels o servicios no cambiados.

- Exposición involuntaria: endpoints o paneles accesibles públicamente.
- Cabeceras de seguridad ausentes: sin CSP, HSTS o X-Frame-Options.
- Entornos mal configurados: permisos excesivos, buckets públicos.
- Inconsistencias entre entornos: staging seguro, producción no.
- Falta de parches: versiones con vulnerabilidades conocidas.
- Exposición de: rutas, claves o stack traces.

Ejemplos típicos

- Credenciales por defecto en bases de datos accesibles.
- Buckets o blobs públicos con datos sensibles.
- Paneles de administración visibles sin autenticación.
- Mensajes de error que revelan información interna.
- Cabeceras, CORS o TLS mal configurados.
- Imágenes de contenedor con herramientas o claves incluidas.
- Versiones obsoletas sin parchear.
- Fallos simples pero explotables en auditorías reales.

¿Qué puede hacer alguien que explote una misconfiguration?

- Acceso a datos o paneles sin autorización.
- Exposición de backups y datos personales.
- Escalada o ejecución remota desde servicios inseguros.
- Movimiento lateral dentro de la red.
- Automatización de ataques al detectar configuraciones débiles.
- Aumenta la superficie de ataque y reduce los costos para el atacante.

¿Cómo detectarlo?

- En despliegue/CI: escanear configuraciones e imágenes con herramientas IaC.
- En pruebas/auditoría: revisar puertos, permisos, cabeceras y versiones.
- En producción: alertas de IAM, logs de acceso anómalo y herramientas CSPM.

Señales típicas: endpoints administrativos públicos, buckets listables, headers ausentes. Comprobaciones periódicas previenen incidentes.

¿Cómo prevenir Insecure Design?

Principios de diseño

- Security by Default: instalar con configuración segura.
- Automatizar IaC con políticas revisadas en PRs.
- Mínimo privilegio en permisos y roles. Separar entornos dev/staging/prod.

Buenas prácticas

- Deshabilitar servicios innecesarios. Cambiar credenciales por defecto, usar vaults.
- Forzar TLS y configurar cabeceras seguras. Revisar CORS en recursos sensibles. Escanear y endurecer imágenes de contenedor.
- Automatizar parches y monitoreo de CVEs. Integrar escáneres de IaC, secretos y dependencias. Centralizar logs y segmentar redes.

Operación continua

- Rotar claves y tokens regularmente. Controlar acceso just-in-time.
- Backups cifrados y privados. Documentar hardening en playbooks.

¿Cómo mitigarlo cuando ya está en producción?

- Identificar recursos y datos expuestos.
- Cerrar acceso (ACLs, IPs, WAF, desactivar servicios).
- Rotar cuentas, claves y secretos afectados.
- Aplicar parches o medidas temporales de bloqueo.
- Eliminar backups públicos y datos filtrados.
- Implementar controles compensatorios (WAF, autenticación).
- Auditar logs para medir el impacto.
- Notificar incidentes según política y corregir laC para prevenir recurrencia.
- Contener primero, remediar después, mejorar proceso siempre.

Checklist operativo rápido

1. ¿Se eliminaron credenciales por defecto?
2. ¿Almacenamiento no es público sin justificación?
3. ¿Endpoints sensibles protegidos y no expuestos?
4. ¿TLS forzado y cabeceras seguras activas?
5. ¿Imágenes de contenedor endurecidas?
6. ¿Permisos IAM mínimos?
7. ¿Escaneos automáticos en CI/repos/IaC?
8. ¿Actualizaciones y CVEs monitoreados?
9. ¿Logs y alertas de cambios activos?

Conclusiones

1. Configuración segura = seguridad real.
2. Son errores fáciles de evitar si hay disciplina de revisión.
3. Como desarrollador junior, impulsa checks automáticos y hardening previo a cada despliegue.
4. Añadir un escáner de secretos o verificador de cabeceras ya es un gran paso.

Cada configuración segura reduce riesgos y demuestra responsabilidad profesional.